0) Objectif global du fichier “images”

Le but de cette partie est de vérifier la **faisabilité de regrouper automatiquement des produits par catégorie** en utilisant **uniquement les images**.
On suit exactement la logique demandée dans le mail + l’exemple Weather :

1. **Prétraitement images**
2. **Extraction de features** (SIFT → BoVW)
3. **Réduction de dimension** (PCA puis T-SNE) pour visualiser
4. **Clustering** (KMeans sur T-SNE ou PCA)
5. **Mesures** (ARI + matrice de confusion + rapport)

1) Chargement du dataset + construction des chemins d’images

Pourquoi ?
Le CSV Flipkart contient un nom de fichier image (colonne image) mais pas forcément le chemin complet.
On construit donc image_path en pointant vers le dossier Images/.

Variables créées/modifiées
- df : DataFrame principal du CSV
- IMAGE_DIR = "Images"
- df["image_path"] : chemin complet vers l’image (ex: Images/xxxx.jpg)
- df["has_image"] : booléen (True si l’image existe réellement sur disque)
- df_img : sous-ensemble ne gardant que les lignes avec images valides

Rôle des index
- df_img.reset_index(drop=True) : on remet un index propre (0..N-1) pour éviter des décalages plus tard.

2) Création des labels (équivalent Weather)

Pourquoi ?
Dans Weather, les labels (cloudy/rain/...) viennent du nom du fichier image.
Chez Flipkart, les classes viennent d’une colonne catégorie dans le CSV.

Problème rencontré
Je n’avais pas toujours main_category dans ton CSV, donc on a fait une détection automatique.

Variables créées
- category_col : la colonne utilisée comme catégorie (ex: product_category_tree)
- df_img["label_name"] : la catégorie texte (classe “humaine”)
- df_img["label"] : encodage numérique (classe “machine”)

Important : différence label_name vs label
- label_name = texte lisible (ex: "Watches" ou chemin complet de catégorie)
- label = entier pour les métriques/modèles (ex: 0..641 si j' encodes une hiérarchie complète)

Ensuite, j'ai créé **main_class** (catégorie de niveau 1) pour retrouver **7 classes** :
- df_img["main_class"] = df_img["label_name"].apply(lambda x: x.split(">>")[0].strip())

C’est l’équivalent “Weather = 4 classes” / “Flipkart = 7 classes”.

3) Affichage d’exemples d’images par catégorie

Pourquoi ?
C’est une étape EDA visuelle comme dans Weather : vérifier que les images existent et comprendre ce qu’on manipule.

Ce qu’on a fait
- Afficher un petit nombre d’images par catégorie **sans charger tout le dataset**.
- On a limité à **3–4 images par catégorie** avec un échantillonnage.

Variables clés
- cats = df_img["main_class"].unique()
- sélection d’images par sample(n=4)

4) SIFT : visualisation des keypoints sur une image

Pourquoi ?
Weather montre une image avec des cercles/keypoints et affiche :
- nombre de descripteurs
- taille des vecteurs (128)

Problème rencontré
Dans certains environnements, cv2.xfeatures2d.SIFT_create() n'existe pas.
Solution :
- utiliser cv2.SIFT_create() (OpenCV moderne)

Variables clés
- sift = cv2.SIFT_create()
- kp : liste des keypoints détectés
- des : matrice des descripteurs SIFT (shape = (N, 128))

Affichage "plus visible"
On a utilisé :
- cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
- possibilité d'augmenter la taille des keypoints (en jouant sur l'image ou en ORB fallback si besoin)

5) Extraction SIFT sur un sous-ensemble d'images

Pourquoi ?
C'est l'étape qui a fait crasher le kernel quand J' essayais sur trop d'images.
SIFT produit énormément de descripteurs → mémoire explose.

Ce qu'on a fait (très proche Weather)
- On traite **un sous-ensemble** (ex: 300 images)
- On stocke :
  - les descripteurs par image
  - une concaténation globale pour créer le "vocabulaire"

Variables importantes
- sift_keypoints_by_img : liste/array d'objets, 1 élément = descripteurs d'une image
- sift_keypoints_all : concaténation de tous les descripteurs (gros tableau)
- valid_paths : liste des chemins des images réellement utilisables (celles qui ont des descripteurs)
- temps1 / duration1 : temps de calcul

Notion d'index (très important)
valid_paths sert à **réaligner** les labels ensuite.
Sans ça : erreurs "length mismatch".

6) MiniBatchKMeans : création du vocabulaire visuel (BoVW)

Pourquoi ?
On transforme des descripteurs (N x 128) en "mots visuels".
C'est exactement Weather : un "vocabulaire" de taille k.

Variables clés
- k = int(round(np.sqrt(len(sift_keypoints_all)), 0))
- kmeans = MiniBatchKMeans(n_clusters=k, ...)
- kmeans.cluster_centers_ : les centres = mots visuels

7) Création des features images = histogrammes BoVW

Pourquoi ?
Une image doit devenir un vecteur fixe pour faire PCA / TSNE / KMeans.
Pour chaque image :
1. kmeans.predict(des) → id cluster pour chaque descripteur
2. histogramme normalisé sur k clusters

Variables clés
- im_features : matrice finale (nb_images, k)
- build_histogram(kmeans, des, image_num) : fonction qui construit l'histogramme d'une image
- image_num (ou i) = index de boucle (identifiant image dans le sous-ensemble)

8) PCA (réduction) puis T-SNE (visualisation 2D)

PCA (comme Weather)
- But : réduire la dimension en gardant 99% de variance
- feat_pca = PCA(n_components=0.99).fit_transform(im_features)

T-SNE (comme Weather)
- But : projection 2D des images
- X_tsne = TSNE(...).fit_transform(feat_pca)
- df_tsne = DataFrame(tsne1, tsne2)

Gros piège rencontré
J' avais :
- im_features construit sur 300 images
- mais df_img avait 1050 lignes
  → mismatch

Solution : construire df_tsne["class"] à partir du même sous-ensemble :
- soit via df_sift réaligné
- soit via une liste valid_labels calculée au moment où je construis im_features

9) Clustering sur T-SNE + mesure ARI
Pourquoi ?
Weather calcule :
- KMeans sur l'espace T-SNE (2D)
- ARI entre clusters et vraies classes

Variables clés
- cls = KMeans(n_clusters=7)
- df_tsne["cluster"] = cls.labels_
- ari = adjusted_rand_score(y_true, df_tsne["cluster"])

Point important :
- ARI doit être calculé avec main_class (7 classes), pas avec label_name (642 classes)

10) Matrice de confusion + heatmap + classification_report

Pourquoi ?
L'ARI donne un score global, mais la heatmap montre *où ça se mélange*.

Variables clés
- labels_true = LabelEncoder().fit_transform(df_tsne["main_class"])
- conf_mat = confusion_matrix(labels_true, df_tsne["cluster"])
- classification_report(labels_true, mapped_clusters) si je fais un mapping cluster→classe dominante

Pourquoi on voyait des matrices "0 et 1" ?
Parce qu'on utilisait **trop de classes** (238 / 642) face à 7 clusters, donc c'était quasi one-hot dispersé.
En repassant à main_class = 7 classes : heatmap lisible.

Conclusion générale

- Les images projetées via BoVW-SIFT + T-SNE montrent une structure, mais la séparation reste limitée.
- Le score ARI obtenu (ex: ~0.08 sur T-SNE) indique un regroupement **faible à modéré** par catégorie principale.
- Certaines catégories sont visuellement proches (ex : "Home Furnishing" vs "Home Decor", etc.), ce qui explique le mélange.
- Cette étape valide la démarche "approche classique" (SIFT/BoVW), et justifie l'étape suivante (CNN Transfer Learning) pour améliorer la représentation.